SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I **Uppu Sye Kumar/192325121** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: U.Sye Kumar **Name of the student:Uppu Sye Kumar**

QUESTIONS:05 **Total Marks:25**

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

Gradient Descent updates model parameters as:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

where:

- θ= parameters
- η= learning rate
- $J(\theta)$= loss function

Effect of a High Initial Learning Rate

- Takes large optimization steps.
- Quickly moves toward the minimum.
- Escapes shallow local regions and plateaus.
- Reduces training time during early iterations.

Effect of Learning Rate Decay

As training progresses, reducing the learning rate:

- Makes parameter updates smaller.
- Prevents oscillation around the optimum.
- Improves convergence accuracy.
- Helps reach the global minimum for convex loss functions.

When It Outperforms a Constant Learning Rate

A decaying learning rate performs better when:

- The dataset is large.
- The loss function is convex.
- Fast initial learning is needed.
- Fine-tuning near the optimum is important.

Possible Risks

- Learning rate too high → overshooting or divergence.
- Learning rate decreases too quickly → premature convergence.
- Learning rate decreases too slowly → unnecessary oscillations.

Conclusion

A high initial learning rate with gradual decay combines **fast initial convergence** and **stable final optimization**, often outperforming a constant learning rate if the decay schedule is chosen appropriately.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:
Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases
  What does this imply about the robustness of MLE?

  Given

  Dataset:

$$x_1, x_2, \dots, x_n \sim N(\mu, \sigma^2)$$

  where:

- Mean (μ) = Unknown
- Variance (σ^2) = Known

Step 1: Likelihood Function

$$L(\mu) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\quad -\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

Step 2: Log-Likelihood

$$\log L(\mu) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Step 3: Differentiate

$$\frac{d}{d\mu}\log L(\mu) = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu)$$

Set derivative to zero:

$$\sum_{i=1}^{n}(x_i - \mu) = 0$$

Therefore,

$$\boxed{\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

Thus, the MLE of the mean is the **sample mean**.

Effect of Outliers

- Outliers significantly influence the sample mean.
- Even one extreme value shifts the estimate.
- Parameter estimate becomes biased.

Effect of Increasing Sample Size

As $n$increases:

- Variance of the estimator decreases.
- Estimate approaches the true mean.
- Accuracy improves.

By the Law of Large Numbers,

$$\hat{\mu} \to \mu$$

Implication on Robustness

MLE for Gaussian mean is:

- Efficient for clean data.
- Consistent as sample size increases.
- **Not robust** to outliers because the mean is highly sensitive to extreme values.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Challenges

Given:

- Features = 10,000
- Samples = 500

Problems:

- High dimensionality.
- Overfitting.
- High computational cost.
- Sparse feature space.

- Poor generalization.

Proposed Algorithm Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize features.

Reason: Ensures all features are on the same scale.

Step 2: Feature Selection

Methods:

- Chi-Square
- Mutual Information
- LASSO
- Recursive Feature Elimination (RFE)

Reason: Removes irrelevant features and reduces dimensionality.

Step 3: Dimensionality Reduction

Use:

- Principal Component Analysis (PCA)

Reason: Retains maximum variance while reducing the number of features.

Step 4: Regularization

Use:

- L1 (LASSO)
- L2 (Ridge)

Reason: Prevents overly complex models and reduces overfitting.

Step 5: Model Training

Use:

- Support Vector Machine (Linear Kernel)
- Logistic Regression with Regularization

Step 6: Cross-Validation

Use:

- K-Fold Cross Validation

Reason: Estimates model performance reliably.

How This Mitigates Overfitting

- Removes noisy features.
- Reduces model complexity.
- Improves generalization.
- Lowers variance.
- Prevents memorization of training data.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Why More Hidden Neurons Increase Representational Power

Each hidden neuron learns a different nonlinear feature.

Output:

$$y = f(\sum w_i x_i + b)$$

Adding more neurons:

- Learns more complex patterns.
- Captures nonlinear relationships.
- Models intricate decision boundaries.

According to the Universal Approximation Theorem, an MLP with enough hidden neurons can approximate any continuous function.

Why More Neurons Do Not Always Improve Performance

Excessive neurons lead to:

- More parameters.
- Higher computational cost.
- Greater risk of memorizing training data.

This results in **overfitting**.

Overfitting

Characteristics:

- Very high training accuracy.
- Low testing accuracy.
- Poor generalization.

Generalization

A well-generalized model:

- Learns underlying patterns.
- Performs well on unseen data.
- Avoids memorizing noise.

Conclusion

Increasing hidden neurons improves model capacity but must be balanced with regularization techniques (Dropout, Early Stopping, L2 Regularization) to achieve good generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

1. Impact on Gradient Updates in Backpropagation

High-dimensional data causes:

- Sparse feature space.
- Noisy gradients.
- Slow and unstable parameter updates.
- Vanishing or exploding gradients in deep networks.

2. Impact on SGD Convergence

SGD updates:

$$w = w - \eta \nabla J(w)$$

In high dimensions:

- Gradient variance increases.
- Updates become noisy.
- More iterations are needed.
- Convergence slows down.

3. Impact on Model Generalization

High-dimensional data leads to:

- Increased overfitting.

- Poor prediction on unseen data.
- High variance.
- Reduced generalization ability.

Techniques to Overcome These Issues

Technique 1: Dimensionality Reduction (PCA)

Benefits:

- Removes redundant features.
- Reduces computational cost.
- Improves convergence.
- Enhances generalization.

Technique 2: Regularization

Examples:

- L1 Regularization
- L2 Regularization
- Dropout

Benefits:

- Controls model complexity.
- Prevents overfitting.
- Improves test accuracy.

Additional Techniques

- Batch Normalization
- Xavier/He Weight Initialization
- Adaptive optimizers (Adam, RMSProp)
- Early Stopping
- Feature Selection

Conclusion

The curse of dimensionality negatively affects gradient computation, slows SGD convergence, and increases overfitting. Applying dimensionality reduction, regularization, proper initialization, and

normalization improves training stability, speeds convergence, and enhances the model's ability to generalize.